

Advanced SQL JOINS - Detailed Corporate Training Guide

Structured for Fresh CS/IT Engineering Graduates | 4–6 Hour Corporate Training Session

SQL JOINS – COMPLETE VISUAL GUIDE

INNER, LEFT, RIGHT, FULL OUTER JOIN

SQL JOINS are used to combine rows from two or more tables, based on a related column between them.

1. SAMPLE TABLES & DATA

Customers Table

customer_id	customer_name	city
1	Rahul	Mumbai
2	Anita	Pune
3	Karan	Delhi
4	Priya	Bangalore

Orders Table

order_id	customer_id	amount
101	1	1200
102	2	4500
103	1	800
104	5	2000

NOTE: customer_id = 5 does not exist in Customers table. (So order 104 has no matching customer)

3. INNER JOIN

Definition: Returns only matching rows that have values in both tables.



SQL Query:

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Result:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500

Use Case: Show customers who placed orders. (Only matching records)

4. LEFT JOIN

Definition: Returns all rows from the left table (Customers) and matching rows from the right table (Orders). If no match, NULL is returned from the right table.



SQL Query:

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id;
```

Result:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
Karan	NULL	NULL
Priya	NULL	NULL

Use Case: Find all customers including those who did not place any orders.

5. RIGHT JOIN

Definition: Returns all rows from the right table (Orders) and matching rows from the left table (Customers). If no match, NULL is returned from the left table.



SQL Query:

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id;
```

Result:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
NULL	104	2000

Use Case: Find all orders including those without any customer record.

6. FULL OUTER JOIN

Definition: Returns all rows when there is a match in either left (Customers) or right (Orders) table records. If no match, NULL is returned on the side that doesn't match.



SQL Query:

```
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
FULL OUTER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Result:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
Karan	NULL	NULL
Priya	NULL	NULL
NULL	104	2000

Use Case: Show all customers and all orders including unmatched records from both sides.

Q1. Customers who placed orders (INNER JOIN)

Task: List customer_name and order_id of customers who placed orders.

```
SQL:
SELECT c.customer_name, o.order_id
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Expected Output:

customer_name	order_id
Rahul	101
Rahul	103
Anita	102

Q2. Customers without orders (LEFT JOIN)

Task: List customer_name who did not place any orders.

```
SQL:
SELECT c.customer_name
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Expected Output:

customer_name
Karan
Priya

Q3. Orders without customers (RIGHT JOIN)

Task: Find orders that do not have any matching customer.

```
SQL:
SELECT o.order_id
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Expected Output:

order_id
104

Q4. All customers and their orders (LEFT JOIN)

Task: Show all customers and their orders (including those with no orders).

```
SQL:
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
ORDER BY c.customer_name;
```

Expected Output:

customer_name	order_id	amount
Anita	102	4500
Karan	NULL	NULL
Priya	NULL	NULL
Rahul	101	1200
Rahul	103	800

Q5. All orders with customer info (RIGHT JOIN)

Task: Show all orders along with customer name (including invalid orders).

```
SQL:
SELECT c.customer_name, o.order_id, o.amount
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id
ORDER BY o.order_id;
```

Expected Output:

customer_name	order_id	amount
Rahul	101	1200
Anita	102	4500
Rahul	103	800
NULL	104	2000

8. KEY TAKEAWAYS

- ✓ INNER JOIN → Matching rows only.
- ✓ LEFT JOIN → All from left + matching from right.
- ✓ RIGHT JOIN → All from right + matching from left.
- ✓ FULL OUTER JOIN → All from both sides.
- ✓ Use LEFT JOIN to include all records from main table.
- ✓ Use RIGHT JOIN to include all records from detail table.

9. COMMON MISTAKES

- ✗ Missing ON condition → results in Cartesian product.
- ✗ Using WHERE with outer join columns turns it into INNER JOIN.
- ✗ Not handling NULL values properly.
- ✗ Using wrong join type for requirement.



10. MINI PROJECT IDEA

Create an E-commerce database with Customers, Orders, Products, Payments tables. Practice all types of JOIN queries to generate business reports.

Learning Objectives

- Understand SQL JOIN concepts clearly
- Differentiate between INNER, LEFT, RIGHT and FULL OUTER JOIN
- Write JOIN queries using real-world business data
- Understand JOIN outputs step-by-step
- Solve interview-level JOIN problems
- Practice hands-on SQL exercises

Sample Database Tables

Customers Table

customer_id	customer_name	city
1	Rahul	Mumbai
2	Anita	Pune
3	Karan	Delhi
4	Priya	Bangalore

Orders Table

order_id	customer_id	amount
101	1	1200
102	2	4500
103	1	800
104	5	2000

INNER JOIN

Definition: Returns only matching rows from both tables.

SQL Query:

```
SELECT c.customer_name,  
       o.order_id,  
       o.amount  
FROM Customers c  
INNER JOIN Orders o  
ON c.customer_id = o.customer_id;
```

Expected Output:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500

Step-by-Step Explanation:

- Only matching records are returned
- Customers without orders are excluded
- Invalid order records are ignored

Real-Life Use Case: Used in E-Commerce systems to display customers who placed orders.

LEFT JOIN

Definition: Returns all rows from LEFT table and matching rows from RIGHT table.

SQL Query:

```
SELECT c.customer_name,  
       o.order_id,  
       o.amount  
FROM Customers c  
LEFT JOIN Orders o  
ON c.customer_id = o.customer_id;
```

Expected Output:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
Karan	NULL	NULL

Priya	NULL	NULL
-------	------	------

Step-by-Step Explanation:

- All customers are displayed
- Customers without orders show NULL values

Real-Life Use Case: Used in CRM systems to identify inactive customers.

RIGHT JOIN

Definition: Returns all rows from RIGHT table and matching rows from LEFT table.

SQL Query:

```
SELECT c.customer_name,
       o.order_id,
       o.amount
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id;
```

Expected Output:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
NULL	104	2000

Step-by-Step Explanation:

- All orders are displayed
- Orders without customers appear with NULL

Real-Life Use Case: Used in audit systems to detect invalid records.

FULL OUTER JOIN

Definition: Returns all matching and non-matching rows from both tables.

SQL Query:

```
SELECT c.customer_name,
       o.order_id,
       o.amount
FROM Customers c
FULL OUTER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Expected Output:

customer_name	order_id	amount
Rahul	101	1200
Rahul	103	800
Anita	102	4500
Karan	NULL	NULL

Priya	NULL	NULL
NULL	104	2000

Step-by-Step Explanation:

- Displays all rows from both tables
- Useful for reconciliation and migration reports

Real-Life Use Case: Used in Banking reconciliation systems.

Hands-On Practice Exercises

Exercise 1: Find all customers and their orders.

```
SELECT c.customer_name, o.order_id
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Exercise 2: Find customers without orders.

```
SELECT c.customer_name
FROM Customers c
LEFT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Exercise 3: Find invalid orders.

```
SELECT o.order_id
FROM Customers c
RIGHT JOIN Orders o
ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;
```

Interview Questions

- Difference between INNER JOIN and LEFT JOIN?
- What is Cartesian Product?
- What happens if ON clause is missing?
- Which JOIN is used most in companies?
- Why do NULL values appear in joins?

Performance Tips

- Use indexes on JOIN columns
- Avoid SELECT * in production systems
- Use aliases for readability
- Filter records using WHERE clause
- Use proper JOIN type based on requirement

Common Mistakes

- Missing ON condition
- Using wrong JOIN columns
- Ignoring NULL handling
- Using FULL JOIN unnecessarily
- Joining huge tables without indexes

Mini Project Assignment

- Create Customers, Orders, Products, and Payments tables
- Insert realistic business data
- Perform all JOIN operations
- Generate business reports
- Find customers without orders
- Generate audit report for invalid orders

Session Completion

Students completing this module will be able to confidently use SQL JOINS in interviews and real-world enterprise projects.